Ders 3 Çalışma Özeti

Ders 3 Çalışma Özeti

Genel Konular

- `typedef`
  - Var olan bir C tipine yeni bir isim verir: `typedef long int int32;` sonra `int32 j;`.
  - `#define` gibi bir ön işlemci komutu değildir, bu yüzden `#` ile başlamaz ve `;` ile biter.
  - En çok yapılarla (struct) kullanılır. Örnek: karmaşık sayı yapısı.
    - `struct complexNumber { float real; float imaginary; };` tanımından sonra her kullanımda `struct complexNumber c1;` yazmak gerekir. Fonksiyon parametrelerinde de aynı uzun yazım tekrarlanır.
    - `typedef struct complexNumber { ... } complex;` ile `complex c1, c2;` yazılabilir.
    - Ayrı satır biçimi: önce `struct complexNumber { ... };`, sonra `typedef struct complexNumber COMPLEX;`. Yeni ad büyük harfle yazıldı, iki biçim de aynı işi görür.
  - Yapıyı kullanan bir fonksiyonun prototipi, yapı tanımından **sonra** yazılmalıdır. Önce yazılırsa tip henüz tanınmaz.
  - `typedef`'i C'yi "Türkçeleştirmek" için (`typedef int tamsayi;`) kullanmak yaygın değildir. Asıl kullanım alanı yapılardır.
- İfadeler (expressions)
  - Sabit ifadeler: `5`, `5 + 6 * 13 / 3.0`. İkincisinde çarpma ve bölme aynı öncelikte, soldan sağa: $6 \cdot 13 = 78$, $78/3.0 = 26.0$, sonuç 31.0.
  - Tam sayı ifadeleri (`int j, k`): `j`, `j / k * 3`, `k - 'a'` (`'a'` 97'dir), `3 + (int)5.0`.
  - Ondalıklı ifadeler: `x / y * 5`, `3 + (float)4`. Operandlardan biri ondalıklıysa ifade ondalıklı olur. Cast her zaman toplamadan önce yapılır: `3 + (float)4` 7.0.
  - İşaretçi ifadeleri (`int *p`): `p`, `p + 1`, `"abc"`. `p + 1` aslında `p + 1 * sizeof(int)` adresidir.
  - Bir satıra tek başına `5;` yazmak geçerli bir ifadedir (bir şey yapmaz).
- Virgül operatörü (tekrar): `return 3,4,5;` 5 döndürür. `return 2,3,4;` örneği derste denendi.
- Operatör sınıflandırması
  - Tekli (unary), ikili (binary), üçlü (ternary). Operatörlerin çoğu ikilidir. Tek üçlü operatör koşul operatörüdür (`?:`).
- Kalan operatörü (`%`)
  - Yalnızca tam sayı operand alır: `6 % 2` geçerli, `6.0 % 2.0` geçersiz.
  - Operandlar negatifse kalanın işareti için "Ders sonrası notlar"a bakın.
  - ANSI standardındaki ilişki: `a == (a / b) * b + a % b`.
- Aritmetik atama operatörleri
  - `m += n + x - y` aslında `m = m + (n + x - y)` demektir.
  - Slayt örneği: `int m = 3, n = 4; float x = 2.5, y = 1.0;` için $n + x - y = 5.5$, $m + 5.5 = 8.5$. `m` int olduğu için 8 olur (kesilir).
  - Diğer slayt örnekleri: `m /= x * n + y`, `n %= y + m`, `x += y -= m` (çalışma sorusu).
- Arttırma/azaltma operatörleri
  - `a = b++;` (a=0, b=1): a 1, b 2 olur. `a = ++b;`: önce b 2 olur, a 2 olur.
  - Sonek arttırmada arka planda geçici bir değişkene eski değer alınır, sonra değişken arttırılır.
  - Slayt örneği (`int j = 0, m = 1, n = -1, s;`):
    - `s = m++ - --j;` → $1 - (-1) = 2$.
    - `s = m += ++j * 2;` (bağımsız olarak) → $++j = 1$, $1 \cdot 2 = 2$, $m = 1 + 2 = 3$, $s = 3$.

    * `s = m++ * m++;` sonucu derleyiciye bağlıdır, sorulmaz.
- `for` döngüsünün işleyişi
  - Başlangıç ataması bir kez yapılır. Sonra koşul kontrol edilir. Doğruysa gövde çalışır, sonra arttırma yapılır, tekrar koşul kontrol edilir.
  - Döngü bitince sayaç değişkeni son değerinde kalır. Örneğin `for(i=0; i<2; i++)` sonrası `i` 2'dir. Bunu sonraki kodda hesaba katmamak sık yapılan bir hatadır.
  - Virgülle iki değişkenli döngü: `for(j=0, k=10; k-j > 0; j++, k--)` 5 kez döner. Sonunda j = 5, k = 5.
  - `for(i=0; i<5; i++)` gövdesi 5 kez çalışır, ama karşılaştırma 6 kez yapılır (son karşılaştırma döngüyü bitirir). Algoritma analizinde döngü tekrarı genellikle karşılaştırma sayısı üzerinden hesaplanır.
- İlişkisel operatörler
  - Sonuç 1 (doğru) veya 0 (yanlış) tam sayısıdır.
  - Sıfır dışındaki her değer doğru kabul edilir: `if(-5)` çalışır, `while(-1)` sonsuz döngüdür.
  - Sayısal analizde sık hata: ondalıklı sayıları `==` ile karşılaştırmak. `while(fark == 0.001)` gibi bir koşul hiçbir zaman beklendiği gibi sağlanmayabilir. Bunun yerine `fabs(beklenen - hesaplanan) < epsilon` gibi bir tolerans kullanılır.
  - Slayt örneği (`int j=0, m=1, n=-1; float x=2.5, y=0.0;`): `j <= m >= n` soldan sağa: `(0 <= 1)` 1, `1 >= -1` 1. Sonuç 1.
- Mantıksal operatörler ve kısa devre
  - `&&` ve `||` birden fazla koşulu birleştirir.
  - Kısa devre: `0 && (...)` sağa bakmadan 0, `1 || (...)` sağa bakmadan 1 verir. `1 && (x < y)` sonucu `x < y`'nin sonucudur.
  - `!` mantıksal değilleme.
- Bit işlemleri
  - Sola kaydırma: `5 << 1` = 10 (2 ile çarpma gibi). Soldan taşan bit kaybolur, sağdan 0 gelir. Bu döndürme (rotation) değildir.
  - Sağa kaydırma: `255 >> 3` = 31 (8'e tam sayı bölmesi gibi).
  - `8 << 10` = 8192 = 2^{13}.
  - Örnek: 9430 = `0x24D6`, 5722 = `0x165A`. `9430 & 5722` = `0x0452` = 1106. `9430 | 5722` = `0x36DE` = 14046.
  - Onaltılığı onluğa çevirme: $0x452 = 2 \cdot 16^0 + 5 \cdot 16^1 + 4 \cdot 16^2 = 1106$.
  - XOR (`^`): bitler farklıysa 1, aynıysa 0. Şifrelemede (anahtarla veriyi karıştırmada) çok kullanılır. `~` bitleri tersler.
- Koşul operatörü (`?:`)
  - `z = (x < y) ? x : y;` koşul doğruysa ilk değer, yanlışsa ikinci değer.
  - Bu örnekte `x == y` durumu ayrıca ele alınmamıştır. Gerekirse operatör iç içe kullanılabilir: `v = (x < y) ? x : ((x == y) ? 1 : y);`. Son dal için önce 0 denendi, sonra `y` yazıldı.
- Bellek operatörleri: `*` ve `&`
  - İkili kullanımda `*` çarpma, `&` bit VE'dir. Tekli kullanımda `*p` işaretçinin gösterdiği değer, `&x` değişkenin adresidir (`scanf`'te kullanılır).
- Kontrol akışı
  - `if` gövdesine süslü parantez koymazsanız yalnızca ilk satır koşula bağlıdır. Alt alta yazılan ikinci satır her durumda çalışır. Tek satır olsa bile her zaman `{ }` kullanın.
  - İç içe `if`: `else` her zaman en yakın önceki `if`'e bağlanır (dangling else). Slaytta üç sayının en küçüğünü döndüren iç içe `if` örneği var.
  - `switch`: tam sayı (ve `char`, çünkü karakter de bir tam sayıdır) ifadeler üzerinde çalışır. String ve ondalıklı sayı kullanılamaz.
    * Her `case` sonuna `break` konmazsa sonraki case'ler de çalışır (fall-through). Hiçbir case eşleşmezse `default` çalışır.
    * Derste `char c = 'A'` ile `case 65` eşleşti.
  - **Kural:** Bu derste `break` yalnızca `switch` içinde kullanılır. `continue`, `goto` ve `switch` dışında `break` dilde vardır ve yerine göre gerekebilir, ama algoritma tasarımını bozduğu için kullanılmaz.
  - `while` koşulu baştan yanlışsa hiç çalışmaz. `do-while` en az bir kez çalışır.
  - Boş ifade (null statement): `while((c = getchar()) == ' ');` gövdesi boş bir döngüdür, boşluk dışında bir karakter girilene kadar okur.
  - `for(; i < 5; )` geçerlidir ama okunurluğu bozar. Programlama hileleri değil, herke-

sin anlayacağı kod yazıyoruz.
  - `if(c == 1);` satırındaki `;` koşulun gövdesini boşaltır, sonraki satır her durumda çalışır.
  - `if(j = 0)` atamadır ve değeri 0 olduğu için hiç çalışmaz. `while(i = 5)` sonsuz döngüdür.
  - İç içe döngüler: çarpım tablosu örneği. İç döngü tamamlanmadan dış döngü ilerlemez.
- Ön işlemci: makrolar
  - Makro bir yer değiştirmedir (substitution). Excel makrosuna benzetildi: kaydedilen bir işlem tekrar çalıştırılır.
  - `#define MENU_YAZ "1-Toplama\n2-Çıkarma\n3-Çarpma"`. Uzun makro satır sonuna `\` konarak devam ettirilir. Makro tanımının sonuna `;` konmaz.
  - Derleme öncesinde makronun geçtiği her yere tanım kopyalanır. Fonksiyon çağrısında olduğu gibi dallanma (jump) ve yığın kullanımı yoktur.
  - Parametreli makrolar:
    * `#define MULT_BY_TWO(a) a+a`: `MULT_BY_TWO(5)` = 10, `MULT_BY_TWO(2*3)` = `2*3+2*3` = 12.
    * `#define SQR(x) x*x`: `SQR(2*3)` = 36, ama `SQR(2+3)` = `2+3*2+3` = 11 (25 değil).
    * `SQR((2+3))` ve `#define SQR1(x) (x)*(x)` ile `SQR1(2+3)` = 25.
    * Kural: makro parametrelerini ve makronun tamamını parantez içine alın.
    * `#define MIN(x,y) ((x)<=(y)?(x):(y))`. Dört sayının en küçüğü: `MIN(MIN(2,6), MIN(3,8))` = 2.
  - Makronun avantajları: fonksiyon çağrısı yükü yoktur, tip kısıtı yoktur.
  - Dezavantajları: her kullanımda kod tekrar kopyalanır (kod uzar), yan etkiler olabilir, hata ayıklayıcıda (debugger) makronun içine girilemez.
  - `#undef` bir makroyu geçersiz kılar.
  - Hazır makrolar: `__LINE__` (satır numarası), `__FILE__`, `__TIME__`, `__DATE__`, `__STDC__`. Kodun hangi noktaya kadar sorunsuz çalıştığını loglamak için kullanılabilir.
  - Koşullu derleme: `#if`, `#elif`, `#else`, `#endif`, `#ifdef`, `#ifndef`, `defined(...)`. Örneğin işletim sistemine göre farklı kod derlemek için.
- Çalışma soruları: `x += y -= m` gibi slayt örnekleri ve üç sayının en küçüğünü bulan `MIN3` makrosu.

Hocanın Özellikle Vurguladığı Kısımlar

- `break` yalnızca `switch` içinde. `continue`, `goto` ve `switch` dışı `break` kullanmayın.
- `if`, `else`, döngü bloklarında tek satır için bile süslü parantez kullanın.
- Döngü bitince sayaç değişkeninin son değerine dikkat edin.
- Ondalıklı sayıları == ile karşılaştırmayın, tolerans kullanın.
- Makro parametrelerini parantez içine alın.
- Yapı kullanan fonksiyon prototiplerini yapı tanımından sonra yazın.

Kısa Tekrar Notları

- `5 + 6 * 13 / 3.0` = 31.0. `3 + (float)4` = 7.0. `'a'` = 97.
- `m += n + x - y` → `m = m + (n + x - y)`, int'e atanınca kesilir.
- `for(i=0;i<5;i++)`: gövde 5, karşılaştırma 6 kez.
- `5 << 1` = 10, `255 >> 3` = 31, `9430 & 5722` = 1106, `9430 | 5722` = 14046.
- `SQR(2+3)` = 11, `SQR1(2+3)` = 25, `MULT_BY_TWO(2*3)` = 12.
- `if(j = 0)` hiç çalışmaz, `while(i = 5)` sonsuz döngü.

Detaylı Açıklamalar

Makro yan etkileri

- Makro argümanı makro içinde iki kez geçiyorsa argümandaki yan etkiler iki kez çalışır. Örnek (derste yazılmadı, `gcc` ile denendi): `int a = 2; int r = SQR1(a++);` satırı `(a++)*(a++)` olur. Bu tanımsız davranıştır. GCC'de r = 6, a = 4 çıktı ve derleyici `-Wsequence-point` uyarısı verdi.

- `MULT_BY_TWO(a) a+a` dışarıdan parantezsiz olduğu için `3 * MULT_BY_TWO(2)` = `3*2+2` = 8 olur (12 değil). Bu yüzden makronun tamamı da parantez içine alınmalıdır: `#define MULT_BY_TWO(a) ((a)+(a))`.

Ders sonrası notlar

- **Kodların kontrolü:** Dersteki örnekler ders sonrasında `gcc` ile derlenip çalıştırıldı. 31.0, `m` = 8, `s` = 2 ve 3, döngü sayıları, bit işlemleri, makro sonuçları ve `MIN(MIN(2,6), MIN(3,8))` = 2 derstekiyle aynıdır.
- **`SQR(2+3)` sonucu:** Derste bir an "13 olacaktır" dendi, sonra sınıftan gelen uyarıyla 11 olduğu söylendi. Doğrusu `2 + 3*2 + 3` = 11'dir.
- **`n %= y + m` satırı:** Slayt örneklerindeki bu satır `float y` ile derlenmez. `%` yalnız tam sayı operand alır, `gcc` "invalid operands to binary %" hatası verdi.
- **Kalan operatöründe negatif sayılar:** Derste "kalanın işareti implementasyona bağlı" dendi. Bu C89 için doğruydu. C99'dan beri tam sayı bölmesi sıfıra doğru kesilir ve kalan bölünenin işaretini alır: `-7 % 3` = -1, `7 % -3` = 1, `-7 / 3` = -2.
- **Hazır makrolar nerede?** Derste `__LINE__` gibi makroların `stdio.h` içinde olduğu söylendi. Bu makrolar hiçbir başlık dosyasında değildir. Derleyicinin ön işlemcisi tarafından önceden tanımlanır (predefined), `#include` olmadan da çalışır.
- **`MIN(x,y,z)` tanımı:** Ekrandaki kodda `#define MIN(x,y)` ve `#define MIN(x,y,z)` aynı isimle yazılmıştı. C'de makrolar aşırı yüklenemez (overload). Aynı isim yeniden tanımlanırsa uyarı veya hata alınır. Üç sayı için ayrı bir isim kullanın. Çalışma sorusunun bir cevabı (derste yazılmadı, test edildi): `#define MIN3(x,y,z) MIN(MIN(x,y),z)`, `MIN3(3,1,7)` = 1.
- **`while` ile tolerans:** Derste "== aslında çıkarma yapıyor" diye anlatıldı. Doğru ifade: ondalıklı sayılar ikilik tabanda çoğu zaman tam gösterilemez (0.1 gibi), bu yüzden hesaplanan iki değer neredeyse eşit olsa bile bitleri farklı olabilir ve == yanlış verir.
- **`switch` ve string:** `switch` içinde `char c[2] = "ab"` gibi bir dizi kullanılamaz. Derleme hatası verir.
- **İç içe `if` sorusu:** Slayttaki `if(a<b) if(a<c) return a; else return c; else if(b<c) return b; else return c;` yapısında `else return c;` iç `if`'e bağlanır. Bu kod üç sayının en küçüğünü doğru döndürür, bütün dallara ulaşılabilir.
- **Not:** İsterseniz bu dersin altyazı (.srt) dosyasını Gemini Notebook gibi bir yapay zeka aracına yükleyerek ders hakkında daha detaylı soru-cevaplar yapabilir ve dersi verimli çalışabilirsiniz.